

MATEMÁTICA COMPUTACIONAL APLICADA

Aula 01

Introdução e apresentação da UC

Lógica formal, estruturas discretas e o código que sustenta tudo o que você vai construir.

VS Code · C# · Git & GitHub

Nesta aula



O que é a UC

O que é a Unidade Curricular e por que ela existe no seu curso.



Organização do semestre

Como as aulas, atividades e a atividade em grupo se encaixam no calendário.



Ferramentas que vamos usar

VS Code, C# e ferramentas online para transformar teoria em código.



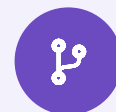
Dúvidas e entregas

Como tirar dúvidas de conteúdo e onde entregar as atividades.

Por que “Matemática Computacional Aplicada”?

Todo sistema que você vai construir na sua carreira — de um app simples a um motor de busca — é, por baixo dos panos, lógica formal e estruturas discretas rodando em silício.

Esta UC existe para te dar o vocabulário e o raciocínio que sustentam tudo isso — e que vão aparecer de novo nas próximas disciplinas do curso.



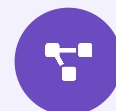
Por que um if funciona

Lógica proposicional



Por que um processador soma números

Álgebra booleana e circuitos digitais



Como redes e mapas viram código

Grafos e árvores



Como isso conecta com o que vem depois

Estruturas de dados e algoritmos

VOCÊ JÁ SABE MAIS DO QUE PENSA

Você já usa isso, mesmo sem saber

Toda vez que você escreve um if com and / &&, está avaliando uma proposição composta — só que em sintaxes diferentes. Esta UC coloca nome e rigor no que você já intui, em qualquer linguagem.

Programa.cs



```
bool estudou = true;
bool dormiuBem = true;

if (estudou && dormiuBem)
{
    Console.WriteLine("Passou!");
}
```

programa.py



```
estudou = True
dormiu_bem = True

if estudou and dormiu_bem:
    print("Passou!")

# mesmo resultado, sintaxe mais curta
```

Mesma lógica proposicional, duas sintaxes: chaves e ; em C# — dois pontos e indentação em Python.

Duas linguagens, a mesma lógica

Você vai ver os mesmos exemplos em Python e em C# ao longo da aula — este é o glossário rápido de tradução entre as duas sintaxes.

CONCEITO



PYTHON



C#

Verdadeiro / falso

`True / False``true / false`

E / OU / NÃO

`and / or / not``&& / || / !`

Bloco condicional

`: e indentação``{ } chaves`

Imprimir na tela

`print("texto")``Console.WriteLine("texto")`

Comentário

`# comentário``// comentário`

Fim da instrução

`quebra de linha``; (ponto e vírgula)`

Dica: volte a este quadro sempre que trocar de exemplo entre as duas linguagens.

Onde cada assunto aparece fora da sala de aula



Lógica proposicional

Regras de validação de formulário — `if (idade >= 18 && cpfValido)`



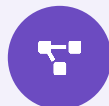
Álgebra booleana

Filtros de banco de dados — `WHERE ativo = true AND saldo > 0`



Circuitos digitais

O processador do seu celular — portas lógicas, literalmente, em silício



Grafos

Rotas do Google Maps, feed de recomendação, rede de amigos



Árvores

Pastas do computador, índice de um banco de dados, o DOM de uma página web

Nenhum desses exemplos é abstrato: cada um é um sistema real que você já usou esta semana.

Como o semestre está organizado

Este trilho segue o calendário real do semestre — incluindo aulas de busca ativa, atividades práticas, orientação de trabalho e avaliações.



Conteúdo

- Lógica proposicional
- Álgebra booleana
- Circuitos digitais
- Grafos e árvores
- Fechamento: estruturas de dados e algoritmos



Prática

- Exercícios de código em C# no VS Code
- Tabelas-verdade verificadas por programação
- Simulação de portas lógicas
- Modelagem de grafos e árvores em código
- Projeto de fechamento conectando tudo



Avaliação

- A1 — avaliação individual
- A3 — atividade aplicada, em grupo
- Tratadas em material didático separado
- Datas e peso: a definir pelo professor

O que você deve conseguir fazer



Construir e interpretar tabelas-verdade.



Simplificar expressões booleanas e projetar circuitos simples.



Modelar problemas do mundo real com grafos e árvores.



Explicar, com suas palavras, por que essas ferramentas aparecem em estruturas de dados e algoritmos do dia a dia.

Ferramentas que vamos usar

Vamos programar em C# para transformar teoria em código verificável.



VS Code

Nosso editor de código no dia a dia da disciplina.



C#

A linguagem que vamos usar para colocar a teoria em prática.



Ferramentas online

dotnetfiddle.net e repl.it — para testar código sem instalar nada.



Git & GitHub

Para versionar seu código e entregar as atividades.

Mais um exemplo: condições encadeadas

 Ingresso.cs

```
int idade = 20;

bool temIngresso = true;

bool acompanhado = false;

// entra se tem ingresso E (maior OU acompanhado)

bool podeEntrar = temIngresso &&

    (idade >= 18 || acompanhado);

Console.WriteLine(podeEntrar); // True
```

 ingresso.py

```
idade = 20

tem_ingresso = True

acompanhado = False

# entra se tem ingresso E (maior OU acompanhado)

pode_entrar = tem_ingresso and (

    idade >= 18 or acompanhado)

print(pode_entrar) # True
```



Teste você mesmo

A mesma fórmula lógica, com parênteses, nas duas linguagens. Rode os dois trechos e mude acompanhado para True (Python) ou true (C#) — no VS Code, no .NET Fiddle ou no repl.it.

Como tirar dúvidas



Antes de perguntar, tente

- Rerler a seção da aula relacionada à sua dúvida.
- Rodar o código de exemplo você mesmo, alterando os valores.
- Formular a dúvida de forma específica (“por que X não dá Y?”), não genérica (“não entendi nada”).



Dúvidas de conteúdo

Procure o professor nos horários de atendimento ou pelo canal oficial da turma.



Dúvidas de entrega

Confira sempre a página da aula correspondente antes de perguntar — o passo a passo costuma estar lá.

Checklist de preparação



Instalar o VS Code e o SDK do .NET (C#)



Criar (ou confirmar) uma conta no GitHub



Rodar o exemplo do if em C# ou Python e testar com estudou = false



Ler a visão geral do cronograma da disciplina

Referências



Livros

ROSEN, K. H. Matemática Discreta e suas Aplicações. 7. ed. — livro-texto de referência do curso.

GERSTING, J. L. Fundamentos Matemáticos para a Ciência da Computação.

SCHEINERMAN, E. R. Matemática Discreta: uma introdução.



Documentação e prática

Microsoft Learn — documentação oficial de C#: learn.microsoft.com/dotnet/csharp

.NET Fiddle — testar C# no navegador: dotnetfiddle.net

Guia de instalação da turma — VS Code + .NET SDK.



Vídeos

Curso em Vídeo (Gustavo Guanabara) — série de Lógica de Programação, ótima para começar do zero.

CS50 (Harvard) — introdução a como computadores representam e processam informação.



PRÓXIMA PARADA

Busca Ativa — Início de Semestre

Vamos acertar ferramentas e dúvidas de ementa.

Depois disso, começamos de verdade com Fundamentos de Lógica.